

Onapookkal • ഓണപ്പൂക്കൾ

Complete architecture & feature guide — how every part of the site works

Storefront <https://onapookkal.store>

Admin /admin

Repo github.com/AanandAB/Onapoo

Generated August 2026

What this document covers

This is a complete reference for the Onapookkal seasonal flower shop — the system architecture, the data model, and a deep walk-through of every customer-facing and admin feature, including exactly how coupons, delivery charging, order placement, the delivery map, and profit & loss work under the hood.

Contents

- | | |
|--|------------------------------------|
| 1. 1. Overview & tech stack | 8. 8. Order lifecycle |
| 2. 2. Deployment architecture | 9. 9. Admin panel (in depth) |
| 3. 3. Data model (D1 schema) | 10. 10. Delivery map |
| 4. 4. Rendering model & hydration safety | 11. 11. Reports & Profit & Loss |
| 5. 5. Storefront features | 12. 12. API routes |
| 6. 6. Checkout, delivery & payments | 13. 13. Security model |
| 7. 7. Coupons (in depth) | 14. 14. Build, deploy & operations |

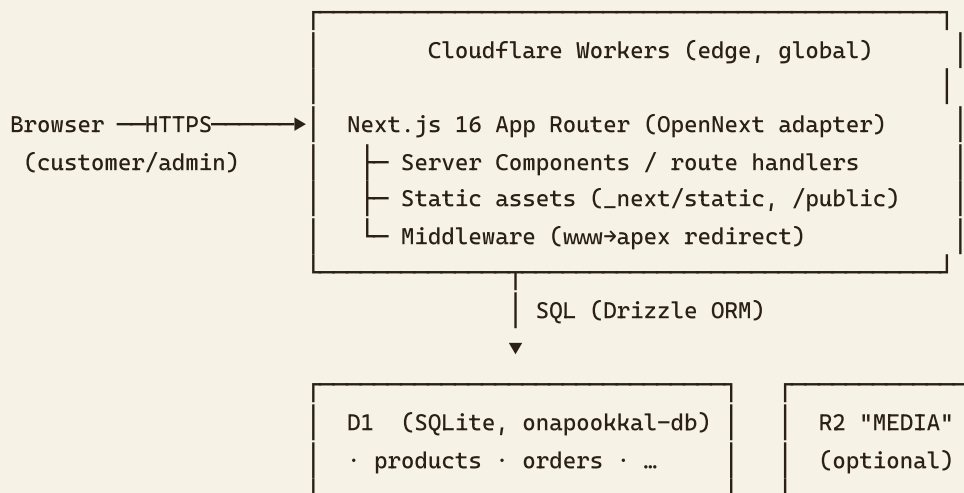
1. Overview & Tech Stack

Onapookkal is a seasonal e-commerce site selling fresh Onam pookalam flowers across Kannur, Kerala. It is bilingual (English + മലയാളം), runs entirely on Cloudflare's edge network, and ships with a full admin/CRM backend so the shop can be run day-to-day without external tooling.

Layer	Technology
Frontend framework	Next.js 16 (App Router) + React 19.2
Language	TypeScript
Styling	Tailwind CSS 4
Hosting	Cloudflare Workers (edge) via @opennextjs/cloudflare 1.20
Database	Cloudflare D1 (SQLite) via Drizzle ORM 0.45
Object storage	R2 (optional — image uploads); images also stored as static assets or base64 in D1
Animation	GSAP 3 + Lenis (smooth scroll)
Icons / map	lucide-react · Leaflet 1.9 (OpenStreetMap)
PDF receipts	jsPDF + html2canvas-pro (client-side)
Fonts	Playfair Display, Plus Jakarta Sans, JetBrains Mono (bundled locally, offline)

Why Cloudflare Workers? The whole app (including Server Components and route handlers) executes at Cloudflare's edge, close to the customer, with zero server management. D1 is a globally-distributed SQLite — the only dependency to keep track of.

2. Deployment Architecture



- **Single Worker** serves both the public storefront (/ , /shop/... , /checkout , /track) and the admin panel (/admin/*).
- **Custom domain** onapookkal.store (+ www alias) with a workers.dev fallback onapookkal.aanandab44.workers.dev .
- **Middleware** does one job: 301-redirect www.onapookkal.store → onapookkal.store (page routes only).
- **D1 binding** env.DB ; **assets binding** env.ASSETS ; optional **R2 binding** env.MEDIA .
- All storefront + admin pages are force-dynamic (server-rendered per request) so prices/stock are always live — no CDN caching of page HTML.

Request lifecycle (a storefront page load)

- 1 Request hits the Worker; middleware redirects www if present.
- 2 The page's async Server Component runs: reads settings , categories , products from D1.
- 3 Server-rendered HTML (with the React Flight payload) is streamed to the browser.
- 4 Client hydrates; client components (cart, i18n, countdown, filters) become interactive.
- 5 Subsequent reads (D1) use a first-primary session so the admin's just-saved changes appear immediately.

3. Data Model (D1 Schema)

Eleven tables, defined in `src/db/schema.ts` with Drizzle ORM. IDs are UUIDs (`crypto.randomUUID()`); timestamps are Unix epochs.

categories

Column	Type	Purpose
<code>id</code>	text (PK)	UUID
<code>slug</code>	text (unique)	e.g. <code>traditional</code> , <code>kits</code>
<code>name_en</code> / <code>name_ml</code>	text	Bilingual display name
<code>color</code>	text	Hex swatch for the filter pill dot
<code>sort_order</code>	integer	Order of pills

products

Column	Type	Purpose
<code>id</code>	text (PK)	UUID
<code>slug</code>	text (unique)	URL slug (<code>/shop/<slug></code>)
<code>name_en</code> / <code>name_ml</code> , <code>color_en</code> / <code>color_ml</code> , <code>description_en</code> / <code>description_ml</code>	text	Bilingual content
<code>category_id</code>	text (FK)	→ categories.id
<code>unit</code>	text	<code>bunch</code> · <code>kg</code> · <code>packet</code> · <code>kit</code> · <code>piece</code>
<code>price</code>	integer	Selling price (₹, whole rupees, per unit)
<code>cost_price</code>	integer	Buying cost (₹/unit) — drives profit
<code>compare_at_price</code>	integer?	Strike-through "was" price → discount %
<code>stock</code>	integer	Available quantity (0 = sold out)
<code>is_featured</code>	boolean	★ + sorts first
<code>hidden</code>	boolean	Hidden from storefront (still in admin)
<code>sort_order</code>	integer	Within-featured ordering
<code>image</code>	text?	Primary image (static path or base64 data URL)
<code>images</code>	json	Gallery array (includes primary)

orders

Column	Type	Purpose
<code>id</code> , <code>order_number</code>	text	UUID + human code <code>ONM-<base36></code>
<code>customer_name</code> , <code>phone</code> , <code>email</code>	text	Contact
<code>address</code> , <code>pincode</code> , <code>district</code> , <code>area</code> , <code>landmark</code>	text	Delivery details
<code>delivery_method</code>	text	<code>delivery</code> · <code>pickup</code>
<code>location</code>	text?	<code>"lat,lng"</code> when the customer shares location
<code>delivery_date</code>	text?	ISO date string
<code>items</code>	json	Array of <code>{productId, name, nameMl, unit, qty, price, costPrice}</code>
<code>subtotal</code> , <code>delivery_charge</code> , <code>discount</code> , <code>total</code>	integer	Money (₹)
<code>coupon_code</code>	text?	Applied coupon, if any
<code>payment_method</code>	text	<code>cod</code> · <code>whatsapp</code> · <code>razorpay</code>
<code>payment_status</code>	text	<code>pending</code> · <code>paid</code> · <code>failed</code>
<code>order_status</code>	text	<code>new</code> → <code>confirmed</code> → <code>packed</code> → <code>out_for_delivery</code> → <code>delivered</code> / <code>cancelled</code>
<code>razorpay_order_id</code> , <code>razorpay_payment_id</code>	text?	Online-payment refs
<code>notes</code>	text?	Free text

Other tables

Table	Key columns	Purpose
<code>coupons</code>	<code>code</code> (PK), <code>type</code> , <code>value</code> , <code>phone</code> , <code>used</code>	Single-use promo codes
<code>offers</code>	<code>title_en/ml</code> , <code>type</code> , <code>value</code> , <code>active</code> , <code>start_at</code> , <code>end_at</code> , <code>product_ids</code>	Percent / flat site offers
<code>admins</code>	<code>username</code> , <code>password_hash</code>	Admin login
<code>settings</code>	<code>key</code> (PK), <code>value</code>	Key/value store (ordering date, announcements, store name...)
<code>expenses</code>	<code>label</code> , <code>amount</code> , <code>created_at</code>	Overhead → P&L
<code>vendors</code>	<code>name</code> , <code>phone</code> , <code>location</code> , <code>supplies</code> , <code>active</code>	Suppliers
<code>purchases</code>	<code>vendor_id</code> , <code>item</code> , <code>quantity</code> , <code>cost</code>	Stock purchases → P&L

4. Rendering Model & Hydration Safety

Pages are **server-rendered per request** (`export const dynamic = "force-dynamic"`), then hydrated on the client. Because the server (Cloudflare, UTC) and the browser (Kannur, IST) can disagree on time and locale, the code deliberately avoids non-deterministic values during render — a class of bug that otherwise causes React hydration mismatches (blank sections, dead buttons).

Patterns used

- **Countdown** — starts as a stable `00:00:00:00` placeholder; `Date.now()` is only called inside a `useEffect` (client-side), so server and client render identically.
- **"Ordering opens" gate** — the open/closed boolean is computed **on the server** in the layout and passed down as a prop; client components never call `new Date()` during render.
- **Price formatting** — a deterministic Indian digit-grouping function (no `Intl` /ICU), so ₹ formatting is byte-identical on server and client.
- **Scroll reveal** — content is visible by default; JS adds a "hidden" class and animates it in (progressive enhancement). If JS fails, the page still shows everything.
- **i18n & cart state** — read from `localStorage` only inside `useEffect` (after hydration), never during render.

5. Storefront Features

5.1 Bilingual UI (English ↔ മലയാളം)

A `LanguageProvider` (React context) holds the current language. All strings live in one `STRINGS` dictionary (`src/lib/i18n.tsx`). The toggle (top-right) flips the language and persists the choice to `localStorage["onam_lang"]` ; the same preference is restored on the next visit. Product content (names, colours, descriptions) is stored per-language in D1, so shop data is translated independently of UI strings.

5.2 Hero & Thiruvonam countdown

An animated pookalam (GSAP: petals assemble on load, parallax on scroll) plus a live countdown to Thiruvonam. Onam 2026: Atham **17 Aug** → Thiruvonam **26 Aug**. The countdown is hydration-safe (see §4).

5.3 Catalog — categories, search, sorting

The catalog (`src/components/catalog.tsx`) renders every visible product in a responsive grid with:

- **Category filter pills** — "All" + one pill per category (with its colour dot). Clicking sets an `active` category; the grid filters client-side on `product.categoryId` .
- **Search box** — filters on `nameEn` , `nameMl` , `colorEn` , `colorMl` (case-insensitive). Search and category combine (AND).
- **Sorting** — featured (★) products first, then `sort_order` .
- **Hidden products** are excluded from the storefront by the D1 queries themselves (not just the UI), so a hidden product also 404s on its detail page.

5.4 Product card

- Image (lazy-loaded) with a fallback glyph; –% discount badge (from `compare_at_price`); ★ featured mark; "Only N left" low-stock warning; sold-out overlay.
- 3D tilt on mouse-move (CSS transform).
- Unit price with `/ kg` , `/ bunch` etc. via `unitLabel` .

5.5 Dynamic kg pricing (grams ↔ rupees)

For `unit ≡ "kg"` products, the card shows a quantity stepper in grams plus two linked controls:

- **Grams → price** — as the customer types grams (or taps –/+), a live line total shows: `500 g = ₹170` (computed as `round(price × qty)` , where `qty` is in kg).
- **₹ → grams** — a "Spend ₹ ___" box reverse-calculates: `grams = round(budget / price × 1000)` , clamped to available stock. ₹340 on a ₹340/kg flower → 1000 g.
- The two fields are two-way synced (a "focused" flag decides which direction is being edited, avoiding input fights).
- Quantities are stored as fractions of a kg in the cart; non-kg products use whole units.

5.6 Cart

A slide-in drawer (`cart-drawer.tsx`) backed by a `CartProvider` . Cart items persist to `localStorage["onam_cart"]` and survive reloads. Quantity can be edited in the drawer; a mobile "view cart" bar and bottom nav keep the cart reachable on phones. Adding an item shows a toast rather than opening the drawer, so customers can keep browsing.

5.7 Ordering window (gate)

Customers can always browse and search, but ordering is blocked until the admin-configured `ordering_start` date (default **21 Aug 2026**). Until then the Add-to-cart button is replaced with "Ordering opens 21 August". The check is enforced server-side in `placeOrder` too, so it can't be bypassed from the browser.

6. Checkout, Delivery & Payments

6.1 Delivery charge (distance-based)

Computed with the Haversine formula from the store's coordinates (11.8314722, 75.5517778):

```
free   ≤ 7 km from the store
₹20 base + ₹5 per extra km beyond 7 km
free   on orders ≥ ₹2,000 (subtotal)
₹30    flat fallback when the customer shares no location
```

The charge is computed **server-side** (never trusts a client number). Pickup orders skip the charge entirely.

6.2 Location & pincode

- **"Share my location"** — captures the browser geolocation as `"lat,lng"`, stored on the order so the delivery map gets an exact pin.
- **Pincode lookup** (`/api/pincode`) — auto-fills district/area from the entered pincode (served from a local lookup).
- Without a shared location, the delivery map falls back to geocoding the pincode.

6.3 Payment methods

- **Cash on Delivery** — order is created with `payment_status: pending`.
- **Pay on WhatsApp** — order is created and a pre-filled WhatsApp message is opened; payment is settled manually.
- **Razorpay (online)** — hidden unless `RAZORPAY_KEY_ID` / `RAZORPAY_KEY_SECRET` are set. The server creates a Razorpay order (amount in paise) and verifies the payment signature with HMAC-SHA256 before marking it paid.

6.4 Order tracking

`/track` lets a customer enter their order number + phone to see a live status timeline and a printable receipt. This is a read-only, customer-facing view of the order's `order_status`.

7. Coupons (in depth)

Coupons live in the `coupons` table and are the site's main promo mechanism.

7.1 Types

Type	value	Effect
<code>percent</code>	1–100	<code>discount = round(subtotal × value / 100)</code>
<code>flat</code>	₹	<code>discount = min(value, subtotal)</code> (never exceeds subtotal)
<code>free_delivery</code>	0	Delivery charge set to ₹0 (no ₹ discount)

7.2 Properties

- **Single-use** — a `used` boolean; once redeemed it can't be used again.
- **Phone-bound** — each coupon is tied to a normalized phone number (digits only, leading `91` stripped). Validation fails if the checkout phone doesn't match.

7.3 Code generation

Codes are 8 characters from an ambiguity-free alphabet (no `0/0/1/I/L`): `ABCDEFGHJKMNPQRSTUVWXYZ23456789`, generated with `crypto.getRandomValues`. Two generators exist: the in-admin generator and a standalone Python GUI (`coupon_generator.py`).

7.4 Redemption flow (checkout)

- 1 Customer types a code + their phone at checkout.
- 2 The client calls `GET /api/coupon?code=...&phone=...&subtotal=...` for a **live preview** — this validates but does **not** mark it used.
- 3 The discount (and/or free-delivery flag) is shown immediately on the totals.
- 4 On submit, `placeOrder` re-validates server-side (phone match, not-yet-used, computes discount from the re-priced subtotal).
- 5 The discount is stored on the order (`discount` , `coupon_code`); the coupon is then marked `used` .

Why re-validate server-side? The preview call could be tampered with. The authoritative discount is always recomputed inside `placeOrder` using the server's own subtotal, so a manipulated client can't mint a bigger discount.

7.5 Coupons in Profit & Loss

Revenue in the P&L is `subtotal - discount` (delivery fees are pass-through, not profit). A dedicated **"Discounts given"** line now surfaces the total coupon discount across all non-cancelled orders, so the owner sees exactly how much was given away.

8. Order Lifecycle

8.1 Placement (server-side `placeOrder`)

- 1 **Gate check** — reject if ordering hasn't opened yet.
- 2 **Input validation** — items present; name + phone; address + pincode for delivery.
- 3 **Re-pricing** — fetch each product from D1 and rebuild the order items from the DB (client prices/qty are ignored). kg quantities are rounded to grams; others floored to whole units.
- 4 **Stock check** — block if a product is sold out or the requested quantity exceeds stock.
- 5 **Compute money** — `subtotal =  $\sum \text{round}(\text{price} \times \text{qty})$` ; apply coupon; compute delivery charge; `total = subtotal - discount + deliveryCharge` .
- 6 **Persist** — insert the order (order number `ONM-<base36 timestamp>`).
- 7 **Reserve stock** — decrement each product's stock by the ordered quantity.
- 8 **Consume coupon** — mark it used (if applied).
- 9 **Razorpay** — if online payment, create the payment order and store its ID.
- 10 **Return** — order summary + a pre-filled WhatsApp message URL to the shop.

8.2 WhatsApp confirmation

Every order opens a pre-filled `wa.me` message to **+91 70340 26295** with the order number, customer details, itemized list (with per-item totals), subtotal, discount, delivery charge and total — bilingual based on the customer's language.

8.3 Status workflow & restock

`new → confirmed → packed → out_for_delivery → delivered` (or `cancelled`). The admin changes status from the orders list. Moving an order **into** "cancelled" automatically **restocks** each item (only on the transition, so re-saving a cancelled order never double-restocks). Manual orders (empty `productId`) never reserved stock and are skipped.

9. Admin Panel (in depth)

9.1 Authentication

- **Password hashing** — PBKDF2-SHA256, 100,000 iterations, random 16-byte salt, stored as `pbkdf2$iterations$salt$hash`. Verification uses a constant-time compare.
- **Session** — a signed token (`payload.signature` , base64url, HMAC-SHA256) with a 7-day expiry, stored in an `httpOnly` cookie `onapookkal_session`.
- **Signing secret** — `AUTH_SECRET`, a Worker secret binding (not `process.env`); a dev fallback is used only when no Cloudflare context exists.
- **Guards** — `requireAdmin()` on every admin page and action; route handlers check `getSession()` and return 401.

9.2 Products

Full CRUD with bilingual names, price / cost / compare-at price, stock, featured, **hidden** (a per-row Hide/Show button + a "HIDDEN" badge), sort order, unit, and images. Images can be a URL, a static path, or a browser-compressed base64 data URL stored in D1 (no R2 required). An optional R2 upload path (`/api/upload` → `/media/<key>`) exists for larger files but is disabled unless the `MEDIA` binding is configured.

9.3 Offers

Percentage or flat site-wide offers with an active flag and optional date range / per-product targeting (`product_ids`).

9.4 Coupons (admin)

List codes with used/unused state, delete, and generate new ones bound to a phone number.

9.5 Orders (CRM)

- Full list with **status filters + search**; change status / payment status inline.
- **Send receipt on WhatsApp** — pre-filled itemized message.
- **PDF bill** — a printable receipt (client-side jsPDF + html2canvas-pro) at `/admin/orders/[id]/receipt`.
- **CSV export** of all orders (`/api/admin/export-orders`).
- **Manual order entry** — for walk-in / phone orders; uses the admin product list (so it can sell hidden products too) and never reserves stock.

9.6 Vendors, purchases, expenses

- **Vendors** — supplier directory (name, phone, location, what they supply, active flag).
- **Purchases** — log of stock bought (vendor, item, quantity, cost).
- **Expenses** — overhead (fuel, packing, labour, rent) that feeds the P&L.

9.7 Settings

Ordering-open date, and the top announcement bar (English + Malayalam). Stored in the key/value `settings` table.

10. Delivery Map (in depth)

`/admin/map` plots every pending home-delivery order on a Leaflet map with OpenStreetMap tiles, so the shop can plan a day's route.

- 1 Fetch all orders where `delivery_method = "delivery"`, then keep those not yet `delivered` or `cancelled`.
- 2 For each order, resolve a coordinate:
 - 3 **Exact** — if the customer shared a location, parse `location` (`"lat,lng"`) and use it directly.
 - 4 **Approximate** — otherwise geocode the `pincode` via Nominatim (OpenStreetMap) and flag the pin as approximate.
- 5 Orders that resolve to a coordinate are plotted as pins (exact vs approximate visual distinction).
- 6 Orders that can't be geocoded (missing/invalid pincode) are listed as "unplotted" alongside the map.

The location share in checkout exists precisely to make these pins exact — pincode geocoding is only a fallback and can be off by neighbourhood-level accuracy.

11. Reports & Profit & Loss

11.1 Reports (/admin/reports)

- Total & today's revenue, pending orders, average order value.
- Order-status breakdown; 14-day revenue chart; top products (by qty); payment-method and delivery-method breakdowns.

11.2 Profit & Loss (/admin/profit)

The core accounting, computed over non-cancelled orders:

```
revenue      = Σ (subtotal - discount)      // money actually received for goods
discounts given = Σ discount                  // total coupons given away
COGS         = Σ (costPrice × qty) per item  // uses the per-order cost snapshot
gross profit  = revenue - COGS
net profit    = revenue - COGS - totalExpenses
```

- **Cost snapshot** — each order item stores its `costPrice` at order time, so profit is frozen even if you later change a product's cost.
- **Expenses** are attributed to the day they were recorded for the per-day net-profit chart.
- **Stock purchases** are shown separately (they're already reflected in COGS, so they're not subtracted a second time).
- The UI renders animated count-up cards (Revenue, Discounts given, COGS, Gross profit, Expenses, Stock purchases, Net profit), a per-day bar chart and a cumulative line chart.

12. API Routes

Route	Method	Purpose
/api/coupon	GET	Live coupon validation/preview (code + phone + subtotal) — does not mark used
/api/pincode	GET	Pincode → district/area lookup
/api/upload	POST	Admin-only image upload to R2 (returns /media/<key>)
/media/[key]	GET	Serve an uploaded media object from R2 (immutable cache)
/api/health	GET	Health check (writes a timestamp to settings)
/api/admin/export-orders	GET	Admin CSV export of orders

Server actions ("use server" functions in `src/app/admin/actions.ts` and `src/lib/order-actions.ts`) handle mutations: login/logout, product save/delete/hide, offers, coupons, order status/payment, manual orders, expenses, vendors, purchases, settings.

13. Security Model

- **Admin auth** — PBKDF2 hashed passwords, HMAC-signed 7-day sessions, httpOnly cookie, `requireAdmin` on every page/action, 401 on unauthorized API calls.
- **Server-side re-pricing** — order totals, prices, delivery charge and stock are recomputed from D1 at order time; the client's numbers are never trusted.
- **Coupon authority** — discounts are re-validated and recomputed server-side; single-use + phone-binding are enforced in `placeOrder`, not just the preview API.
- **Secrets** — `AUTH_SECRET` and Razorpay keys are Worker secrets, never in the repo.
- **Public repo** — the repository is public (`github.com/AanandAB/Onapoo`); no credentials, hashes, or the admin password may be committed.
- **Canonical host** — middleware 301-redirects `www` → apex for a single canonical origin.

14. Build, Deploy & Operations

Local development

```
npm install
npm run db:generate      # Drizzle migration from schema
npm run db:migrate:local # apply migration to local D1 (miniflare)
npm run db:seed:local    # seed categories, products, admin, settings
npm run dev              # http://localhost:3000
```

Deploy (manual — no CI/CD)

```
npm run build    # next build (Turbopack)
npm run deploy   # opennextjs-cloudflare build + deploy to Cloudflare
```

Kill any running `next dev` first — it holds a lock on `.open-next` (EPERM).

Secrets

Secret	Purpose
<code>AUTH_SECRET</code>	Session-signing secret (<code>wrangler secret put AUTH_SECRET</code>)
<code>RAZORPAY_KEY_ID</code> / <code>RAZORPAY_KEY_SECRET</code>	Optional — enable online payments

D1 schema changes

Adding a column/table is a two-step job: edit `src/db/schema.ts`, generate a migration, and apply it with `wrangler d1 migrations apply onapookkal-db --remote` (or an ad-hoc `wrangler d1 execute ... --command` / `--file`).

Free-plan headroom (as of Aug 2026)

- D1: ~176 KB used of 5 GB; ~13K rows read/day of 5M limit.
- Workers: ~1.1K D1 reads/day vs 100K requests/day free.
- Static assets: a few MB of 5 GB.
- Comfortably within the free tier, even at Onam launch.